

RESUME ANALYZER

Project Documentation

Project Type	AI-assisted Resume Analysis System
Backend	Django / Django REST Framework
Frontend	Next.js / React
Authentication	Django authentication with JWT
Database	Relational database using Django ORM

This document describes the purpose, requirements, architecture, data model, processing flow, implementation, testing approach, limitations, and future enhancements of the Resume Analyzer project.

1. Introduction

The Resume Analyzer is a web-based system designed to analyze uploaded resumes against a selected job role and job description. The system extracts information from resumes, evaluates relevant skills and keywords, considers required experience, and produces a structured analysis for each uploaded resume.

The project is intended to make resume screening faster and more systematic. Instead of manually reviewing every resume against a job requirement, the system provides measurable scores, matching information, missing skills, suggestions, and rankings that help a user compare candidates.

1.1 Purpose

- Provide a centralized platform for uploading and analyzing resumes.
- Compare resumes with a selected role and job description.
- Identify matched and missing technical skills.
- Evaluate relevant keywords and text similarity.
- Consider required experience when calculating the overall result.
- Store analysis results so they can be viewed later.
- Rank analyzed resumes to support candidate comparison.

1.2 Scope

The system covers resume upload, text extraction, resume analysis, scoring, result storage, result retrieval, and result presentation through a web interface. It also supports authentication so analysis results can be associated with the appropriate Django user.

2. Problem Statement

Recruiters and users may receive many resumes for a single role. Manual comparison can be time-consuming and can make it difficult to consistently evaluate skills, keywords, experience, and relevance to a job description. A resume may contain useful information but still have important gaps for the target position.

The Resume Analyzer addresses this problem by processing resumes against defined job requirements and presenting the results in a structured form. The system helps the user quickly identify stronger matches and understand which skills are missing from a particular resume.

3. Project Objectives

- Develop a web-based resume analysis platform.
- Allow authenticated users to upload resumes for analysis.
- Support common resume file formats used by the project.
- Accept a role and job description as analysis requirements.
- Extract and normalize resume text.
- Calculate skill, experience, keyword, and text-similarity related measures.
- Generate a total resume score and match percentage.

- Identify missing skills and provide learning-oriented suggestions where available.
- Rank analyzed resumes for comparison.
- Persist analysis results in the database.
- Provide a dedicated interface for viewing detailed results.

4. Users and System Requirements

4.1 Main User

The main actor is the authenticated system user. The user can sign in, upload resumes, provide or select job requirements, submit resumes for analysis, and view stored analysis results.

4.2 Functional Requirements

- User authentication and authorization.
- Resume file upload.
- Multiple-resume analysis workflow.
- Role and job-description input.
- Resume text extraction.
- Skill matching.
- Keyword matching.
- Experience evaluation.
- Text similarity evaluation.
- Score and match-percentage generation.
- Missing-skill identification.
- Suggestions for selected missing skills.
- Resume ranking.
- Persistent storage of analysis results.
- Viewing an individual resume analysis by its identifier.

4.3 Non-Functional Requirements

- Usability: results should be understandable and clearly presented.
- Security: protected endpoints should require valid authentication credentials.
- Maintainability: frontend and backend responsibilities should remain separated.
- Scalability: the architecture should allow additional analysis techniques and roles to be added.
- Reliability: failed analysis of one resume should be represented through the stored error field rather than silently losing the result.

5. System Architecture

The application follows a client-server architecture. The Next.js/React frontend provides the user interface and communicates with the Django backend through HTTP requests. Django REST endpoints handle authentication, file processing, analysis, ranking, and retrieval of stored results.

Django ORM manages persistence in the configured relational database.

Layer	Responsibility
Presentation	Next.js / React pages and components for authentication, upload, result listing, and detailed result viewing.
API / Application	Django REST endpoints for authentication, resume upload, analysis, and result retrieval.
Analysis	Text extraction, skill/keyword matching, experience scoring, similarity evaluation, scoring, and ranking logic.
Data	Django ORM models and database records, including the Django User and ResumeResult entities.
File Storage	Uploaded resume files stored through Django FileField/media storage.

6. Database Design

The current database model shown in the project contains the Django User entity and the ResumeResult entity. Each ResumeResult belongs to one User through a foreign-key relationship. The relationship uses cascade deletion, meaning that a user's stored resume results are removed when that user is deleted.

6.1 Django User

The project uses Django's built-in User model for authentication and ownership of resume analysis records. The User model is not redefined in the project model shown; it is imported from `django.contrib.auth.models`.

6.2 ResumeResult

Field	Type	Purpose
user	ForeignKey	Owner of the analysis result; related to Django User.
filename	CharField	Original/display name of the uploaded resume.
file	FileField	Uploaded resume file stored under the resumes/upload path.
url	URLField	Optional URL associated with the uploaded file.
role	CharField	Job role used for the analysis.
description	TextField	Job description used as an analysis requirement.
required_experience	PositiveIntegerField	Required experience in years.

Field	Type	Purpose
text_info	JSONField	Complete structured analysis generated by the resume analysis logic.
rank	PositiveIntegerField	Ranking assigned to the result; nullable.
score	FloatField	Overall resume score.
match_percentage	FloatField	Calculated match percentage.
top_missing_skill	CharField	Most important/missing skill identified for the result.
error	TextField	Error information if analysis of this resume fails.
created_at	DateTimeField	Automatically records result creation time.

6.3 Entity Relationship Description

Django User (1) → (Many) ResumeResult: One authenticated user can have many resume analysis results, while each ResumeResult belongs to one user. The foreign key uses `related_name="resume_results"` and `on_delete=models.CASCADE`.

7. System Diagrams

The project documentation includes three core design diagrams: the Use Case Diagram, Entity Relationship (ER) Diagram, and Data Flow Diagram (DFD). These diagrams should be inserted in this chapter in the final academic submission.

7.1 Use Case Diagram

The Use Case Diagram represents interactions between the system user and the Resume Analyzer. Major use cases include registration/login, uploading resumes, entering job requirements, analyzing resumes, viewing results, viewing detailed analysis, and accessing stored resume results.

7.2 ER Diagram

The ER Diagram represents the database relationship between Django's User entity and ResumeResult. The key relationship is one-to-many: a single user can own multiple analysis results.

7.3 Data Flow Diagram

The DFD represents how information moves through the system. The user provides authentication information, resumes, and job requirements. The application processes the files and requirements, performs analysis, stores the resulting information, and returns analysis results to the user interface.

8. Resume Analysis Workflow

1. Authentication

The user authenticates and receives credentials used for protected API requests.

2. Resume Selection

The user selects one or more resume files through the frontend.

3. Job Requirements

The role, job description, and required experience are supplied for the analysis.

4. Upload Request

The frontend sends the selected files and job requirements to the Django backend.

5. Text Extraction

The backend extracts readable text from the uploaded resume.

6. Resume Processing

The extracted text is processed for relevant skills, keywords, experience information, and similarity.

7. Scoring

The system calculates analysis measures and produces a total score and match percentage.

8. Missing Skills

Skills that are not found are identified and suggestions may be attached.

9. Ranking

Analyzed resumes can be assigned ranks for comparison.

10. Persistence

The result is saved as a ResumeResult associated with the authenticated Django User.

11. Presentation

The frontend retrieves and displays the stored results, including detailed information for an individual resume.

9. Analysis and Scoring

The returned analysis structure demonstrates several scoring components. ResumeScores can contain SkillScore, ExperienceScore, KeywordScore, TotalResumeScore, and DescriptionScores. DescriptionScores includes matched skills, missing skills, matched keywords, a skills score, a keyword score, and a TF-IDF cosine score.

The system also stores SkillsRequiredScores. This structure contains the score obtained from required skills, matched skills, unmatched skills, and MissingSkillWithSuggestions. This allows the detailed result page to show not only a numeric result but also an explanation of skill gaps.

Analysis Component	Purpose
Skill Score	Represents the contribution from recognized technical skills.

Analysis Component	Purpose
Experience Score	Represents how the resume's experience compares with the required experience.
Keyword Score	Represents relevant keyword matching against the target requirements.
TF-IDF Cosine Score	Represents text-level similarity between resume content and the target description.
Total Resume Score	Overall score stored for the resume.
Match Percentage	Percentage-based representation of relevant skill matching.
Ranking	Position assigned among analyzed resumes when ranking is performed.

10. Backend and API Design

The backend is implemented with Django and Django REST Framework. The frontend communicates with the backend using HTTP requests. Protected requests include an authorization header containing the access token.

Area	Responsibility
Authentication	User authentication and JWT token handling.
Resume Upload / Analysis	Accepts resume files and job requirements, then performs analysis.
Result Listing	Returns stored resume analysis results for the authenticated user.
Result Detail	Returns a specific resume analysis record identified by its ID.
Media/File Access	Provides access to uploaded resume files through configured media storage.

11. Frontend Design

The frontend is implemented with Next.js and React. Client-side pages use React state and effects where browser-only information such as localStorage access is required. The interface provides authentication-related pages, resume upload functionality, a results list, and a detailed resume analysis page.

11.1 Resume Results Page

The results page retrieves analysis results for the authenticated user and displays the total number of resumes, best score, best match, ranking, filename, role, score, match percentage, missing skill, date, and a View action.

11.2 Resume Detail Page

The detail page receives the selected resume ID from the route, requests the corresponding analysis from the backend, and displays the filename, role, resume score, rank, match percentage, required experience, top missing skill, job description, and a link to the uploaded resume.

12. Security Considerations

- Protected API requests use JWT-based authentication.
- Resume results are associated with the authenticated Django User.
- The backend should verify ownership before returning an individual result.
- Uploaded files should be validated for supported extensions and reasonable size limits.
- Sensitive authentication tokens should be handled carefully and should not be logged.
- Production deployments should use HTTPS and secure environment variables for secrets and API keys.

13. Testing Strategy

Testing should cover both the backend analysis workflow and the frontend user experience. The following test cases represent the main areas to verify before final submission.

Area	Test	Expected Result
Authentication	Valid login	Authenticated user receives a valid access token and can access protected resources.
Authentication	Invalid/expired token	Protected API rejects the request.
Upload	Single resume	File is accepted and analyzed.
Upload	Multiple resumes	All selected files are submitted and processed as intended.
Analysis	Matching skills	Skills present in the resume are identified.
Analysis	Missing skills	Skills absent from the resume are identified.
Experience	Required experience	Experience score changes according to the configured requirement.
Results	Result listing	Only the authenticated user's appropriate results are returned.
Detail	Valid result ID	The selected result is displayed.
Detail	Invalid result ID	A suitable not-found/error response is displayed.
Failure handling	Unsupported/corrupt file	The failure is handled without crashing the complete request workflow.

14. Current Limitations

- Resume quality and formatting can affect text extraction and therefore the quality of analysis.
- Skill matching depends on the configured skill vocabulary and matching logic.
- A keyword match does not necessarily prove practical proficiency in a technology.
- TF-IDF similarity provides text similarity but does not fully understand the meaning of every sentence.
- The ranking is only as meaningful as the scoring criteria used by the system.
- The current data model shown for the analysis result is centered around ResumeResult and the Django User; additional entities may be required if recruiter, job-post, or organization workflows are added.

15. Future Enhancements

- Add recruiter-specific dashboards and candidate management.
- Create separate Job or JobDescription entities so multiple job postings can be stored independently.
- Add semantic embeddings for stronger meaning-based matching.
- Improve resume section detection for education, projects, certifications, and employment history.
- Add candidate comparison charts and analytics.
- Introduce pagination, filtering, and sorting for large result sets.
- Add background task processing for large batches of resumes.
- Add automated email notifications when analysis is completed.
- Improve file security with stricter validation and malware scanning.
- Provide explainable scoring so users can see exactly how each component contributed to the final score.

16. Conclusion

The Resume Analyzer provides an end-to-end workflow for authenticated resume analysis. It combines a modern web frontend with a Django backend and a structured database model. The system accepts resumes and job requirements, processes the resume content, calculates multiple analysis measures, identifies missing skills, stores the result, and presents the information in a user-friendly interface.

The current design provides a strong foundation for extending the project into a broader recruitment platform. The existing Use Case, ER, and DFD diagrams document the main interactions, data relationships, and information flow, while the database model provides persistent ownership of results through Django's built-in User model.

17. Appendix — Suggested Documentation Order

- 1. Title Page
- 2. Abstract / Executive Summary
- 3. Introduction
- 4. Problem Statement
- 5. Objectives

- 6. Scope
- 7. Requirements
- 8. System Architecture
- 9. Use Case Diagram
- 10. Data Flow Diagram
- 11. ER Diagram
- 12. Database Design
- 13. Backend/API Design
- 14. Frontend Design
- 15. Resume Analysis Methodology
- 16. Scoring and Ranking
- 17. Security
- 18. Testing
- 19. Results / Screenshots
- 20. Limitations
- 21. Future Enhancements
- 22. Conclusion
- 23. References

Note for final academic submission: Insert your actual Use Case Diagram, ER Diagram, and DFD images in Section 7. Add screenshots of the login page, resume upload page, results page, and detailed analysis page in the Results/Screenshots section.